

## STUDENT CASE STUDY EXECUTION REPORT

|                |                             |
|----------------|-----------------------------|
| Department     | FED                         |
| Subject        | DSA-2                       |
| Academic Year  | I-III Semester (2025-26)    |
| Faculty Name   | Mohammed Razia Alamgir Banu |
| Student Name   | A.Shashank REddy            |
| Roll Number    | 2520090139                  |
| Branch         | CSE-IT                      |
| Course Outcome | COURSE OUTCOME - 1          |

**Case Study Topic:** BST and AVL Tree Indexing for an E-commerce Product Catalog Search Index.

### Work Done Summary

Implemented a BST and AVL Tree using product IDs and analyzed search performance. Inserted all given IDs, maintained AVL balance factors within  $\{-1, 0, +1\}$ , and verified that the balanced AVL tree provides efficient  $O(\log n)$  search operations while meeting the search SLA requirement.

### Implementation Details

1. Created a Binary Search Tree (BST) using the given product IDs.
2. Inserted product IDs in the order: 55, 35, 75, 25, 45, 65, 85, 40, 50, 60, 70, 80, 90.
3. Applied AVL balancing rules after each insertion.
4. Calculated the balance factor of every node to ensure it remained within  $\{-1, 0, +1\}$ .
5. Performed necessary rotations whenever imbalance occurred.
6. Maintained a balanced AVL tree for efficient searching.
7. Evaluated search performance using a pointer-hop cost of 180 ns.
8. Verified that search operations satisfy the SLA requirement of less than 4 ms.

### Output

AVL Tree constructed successfully.

Root Node: 55

Height: 3

All nodes satisfy AVL balance conditions.

Search for Product ID 70 completed in 4 pointer hops (720 ns).

Search SLA ( $< 4$  ms) successfully met.

## Result

1. Successfully constructed the AVL Tree using the given product IDs.
2. Maintained the AVL balance property with all balance factors within  **$\{-1, 0, +1\}$** .
3. Achieved a balanced tree height of **3**, ensuring efficient search operations.
4. Retrieved Product ID **70** through the path  **$55 \rightarrow 75 \rightarrow 65 \rightarrow 70$**  with only **4 pointer hops**.
5. Obtained a search time of approximately **720 ns**, which is well below the required **4 ms SLA**.
6. Demonstrated that the AVL Tree provides efficient  **$O(\log n)$**  search performance and is suitable for large-scale e-commerce product indexing.

## Time Complexity

| Operation        | B+ Tree     |
|------------------|-------------|
| Insert           | $O(\log n)$ |
| Search           | $O(\log n)$ |
| Delete           | $O(\log n)$ |
| Traversal        | $O(n)$      |
| Find Max/<br>Min | $O(\log n)$ |

## Conclusion

The AVL Tree proved to be an efficient indexing structure for the e-commerce product catalog. By maintaining a balanced tree, it ensured  **$O(\log n)$**  search, insertion, and deletion operations, resulting in fast and reliable product retrieval. The search for Product ID 70 required only 4 pointer hops and completed well within the 4 ms SLA. Compared to a regular BST, the AVL Tree prevents performance degradation caused by skewed structures, making it a scalable and effective solution for large-scale product indexing and search applications.

**GitHub Link:** <https://github.com/shashankreddy-21/DSA-Project>

| Student Signature | Faculty Signature |
|-------------------|-------------------|
|                   |                   |